



中山大學
SUN YAT-SEN UNIVERSITY

本科生实验报告

实验课程: 操作系统原理实验

任课教师: 刘宁

实验题目: 从实模式到保护模式

专业名称: 计算机科学与技术

学生姓名: 林宏宇

学生学号: 23320093

实验地点: 实验中心 D501

实验时间: 2025 年 3 月 26 日

Section 1 实验概述

学习到如何从 16 位的实模式跳转到 32 位的保护模式，然后在平坦模式下运行 32 位程序。同时，学习到如何使用 I/O 端口和硬件交互，为后面保护模式编程打下基础。

Section 2 预备知识与实验环境

该节总结实验需要用到的基本知识，以及主机型号、代码编译工具、重要三方库的版本号信息等。

- 预备知识：x86 汇编语言程序设计、Linux 系统命令行工具
- 实验环境：
 - 虚拟机版本/处理器型号：Virtualbox
 - 代码编辑环境：Vscode+nasm+C/C++插件
 - 代码编译工具：gcc/g++（64 位）
 - 重要三方库信息：

Section 3 实验任务

- 实验任务 1：课后思考题第 9 题，复现“加载 bootloader”这一节，说说你是怎么做的并提供结果截图
- 实验任务 2：在实验任务一的基础上完成课后思考题第 10 题。
- 实验任务 3：完成课后思考题第 11 题
- 实验任务 4：在进入保护模式后，按照如下要求，编写并执行一个自己定义的 32 位汇编程序，要求简单说一说你的实现思路，并提供结果截图。

Section 4 实验步骤与实验结果

----- 实验任务 1 -----

- 任务要求：

加载 bootloader，在本节中，我们将 lab2 中输出 Hello World 部份的代码放入到 bootloader 中，然后在 MBR 中加载 bootloader 到内存，并跳转到 bootloader 的起始地址执行。

- 思路分析：

本实验的目标是将 lab2 中的代码转移到 bootloader 中，并通过 MBR 加载到内存。

注意事项：

如何通过 MBR 读取并加载 bootloader 到指定内存地址 (0x7e00)。
确保代码能够正确执行，特别是跳转到 bootloader 的起始地址。
在调试过程中要注意扇区的加载顺序、偏移量以及磁盘映像文件的格式。

● 实验步骤:

1. 编写 bootloader.asm 文件

新建一个文件 bootloader.asm, 然后将 lab2 的 mbr.asm 中输出 Hello World 部份的代码, 放入 bootloader.asm

2. 编写 mbr.asm 文件

在 mbr.asm 处放入使用 LBA 模式读取硬盘的代码, 然后在 MBR 中加载 bootloader 到地址 0x7e00

3. 编译.asm 文件格式为.bin 文件

重新编译和写入硬盘起始编号为 0 的扇区

```
nasm -f bin mbr.asm -o mbr.bin
```

4. 编译 bootloader.asm

写入硬盘起始编号为 1 的扇区。

```
nasm -f bin bootloader.asm -o bootloader.bin
```

5. 创建磁盘、将 mbr.bin 和 bootloader.bin 文件写入

使用 dd 命令创建硬盘映像文件 hd.img, 并将 mbr.bin 和 bootloader.bin 分别写入相应的扇区。确保每个文件都正确写入到预定的磁盘位置, 避免数据丢失。

```
dd if=bootloader.bin of=hd.img bs=512 count=5 seek=1 conv=notrunc
```

```
dd if=mbr.bin of=hd.img bs=512 count=1 seek=0 conv=notrunc
```

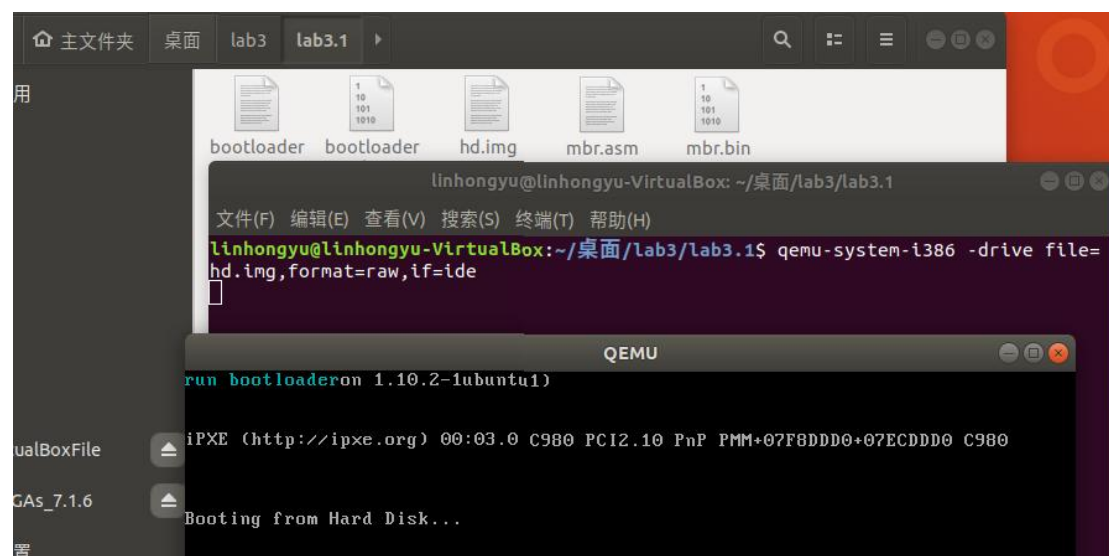
6. 启动 qemu 运行、观察结果

使用 QEMU 启动虚拟机, 并加载硬盘映像 hd.img, 观察是否能够正确输出 "run bootloader".

```
qemu-system-i386 -drive file=hd.img,format=raw,if=ide
```

● 实验结果展示:

上述步骤生成的文件和终端运行指令如下, 且成功观察到了结果输出 root bootloader



----- 实验任务 2 -----

● 任务要求:

在“加载 bootloader”一节中，我们使用了 LBA28 的方式来读取硬盘。此时，我们只要给出逻辑扇区号即可，但需要手动去读取 I/O 端口。然而，BIOS 提供了实模式下读取硬盘的中断，其不需要关心具体的 I/O 端口，只需要给出逻辑扇区号对应的磁头（Heads）、扇区（Sectors）和柱面（Cylinder）即可，又被称为 CHS 模式。现在，同学们需要将 LBA28 读取硬盘的方式换成 CHS 读取，同时给出逻辑扇区号向 CHS 的转换公式。

● 思路分析:

在将 LBA28（逻辑块地址）转换为 CHS（柱面、磁头、扇区）模式时，核心任务是根据 LBA 值计算出相应的 CHS 格式。LBA28 是一个 32 位的地址值，指向磁盘上的某个特定扇区。相比之下，CHS 模式是通过柱面（Cylinder）、磁头（Head）、和扇区（Sector）来定位特定扇区的。

LBA28 使用的是一个逻辑块地址（在 32 位空间中）来指向硬盘的某个扇区。LBA 到硬盘的物理寻址通常需要硬盘的具体物理结构，如每磁道的扇区数和每柱面的磁头数等。

CHS 模式则通过柱面、磁头和扇区的组合来表示硬盘上的一个扇区。转换过程中，我们不需要关心 I/O 端口和具体的物理硬件接口，而是通过 int 13h BIOS 中断来完成硬盘数据读取。

● 实验步骤:

1. 逻辑扇区号向 CHS 的转换公式

CHS 是一种读取硬盘的方式，需要给出三个位置：磁头，扇区和柱面。

LBA28 转 CHS 的转换公式如下：

$$\text{扇区号} = (\text{LBA} \% \text{每磁道扇区数}) + 1$$

$$\text{磁头号} = (\text{LBA} / \text{每柱面扇区数}) \% \text{磁头总数}$$

$$\text{柱面号} = (\text{LBA} / \text{每磁道扇区数}) / \text{磁头总数}$$

在这次实验中，逻辑扇区的 LBA 是 1，每柱面扇区数是 1134

关键参数：

参数	数值
驱动器号（DL 寄存器）	80h
每磁道扇区数	63
每柱面磁头数（每柱面总的磁道数）	18

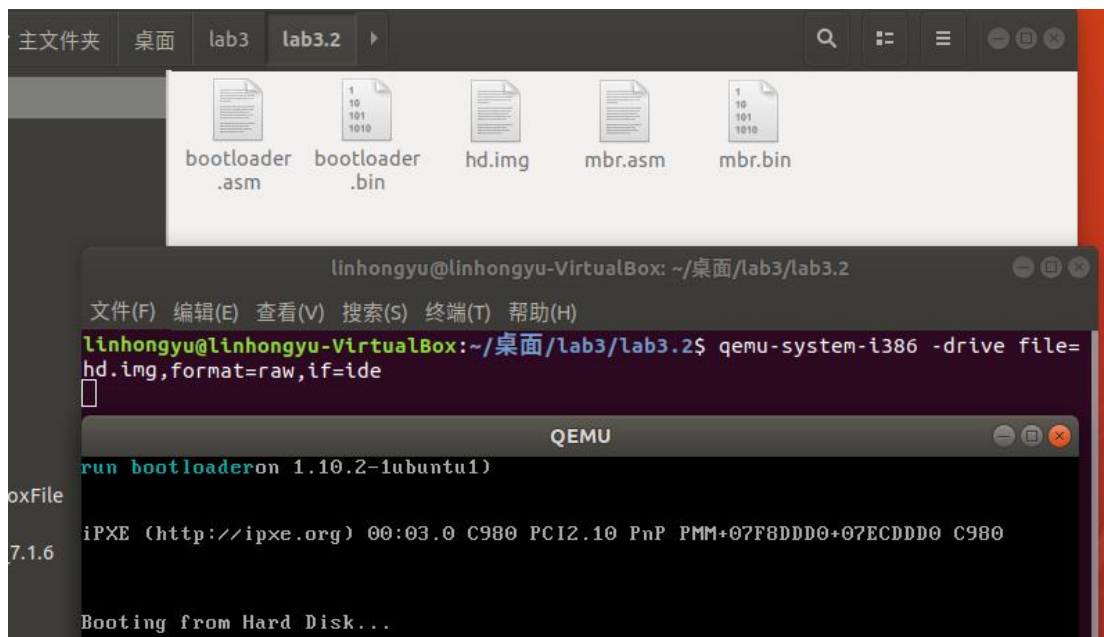
2. 重现编写 asm_read_hard_disk 函数代码

```
asm_read_hard_disk:
    mov ah, 0x02    ;功能号 0x02 -> 读取扇区
    mov al, 1       ;读取 1 个扇区
    mov ch, 0       ;柱面号为 0
    mov cl, 2       ;扇区号为 2
    mov dh, 0       ;磁头号为 0
    mov dl, 0x80    ;驱动器号为 0x80, 表示第一个硬盘
    int 0x13        ;调用 BIOS int 13h 进行硬盘操作
    add bx, 512     ;更新缓冲区地址, 增加 512 字节
    ret
```

3. 执行同上述实验任务一相同的步骤

- 实验结果展示：通过执行前述代码，可得下图结果。

结果同实验任务一相同



----- 实验任务 3 -----

- 任务要求:

复现“进入保护模式”一节，使用 gdb 或其他 debug 工具在进入保护模式的 4 个重要步骤上设置断点，并结合代码、寄存器的内容等来分析这 4 个步骤，最后附上结果截图。gdb 的使用可以参考 appendix 的“debug with gdb and qemu”部份。

- 思路分析:

在进入保护模式之前，理解保护模式的基本概念及调试步骤非常重要。保护模式允许 CPU 启用内存保护、分页、虚拟内存等高级特性。在此实验中，我们需要通过一系列操作，将计算机从实模式转换到保护模式。

使用 GDB 和 QEMU 在关键步骤上设置断点，观察并分析寄存器和内存的变化。

- 实验步骤: 给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. mbr.asm 代码、编写 bootloader.asm 代码编写、boot.inc 代码不再重复实验内容

2. 编译 nasm，生成可重定位文件 mbr.o，-g 是添加 debug 信息

```
nasm -o mbr.o -g -f elf32 mbr.asm
```

3. 为可重定位文件 mbr.o 指定起始地址 0x7c00，链接生成可执行

```
ld -o mbr.symbol -melf_i386 -N mbr.o -Ttext 0x7c00
```

4. 为 bootloader 重复相同步骤

```
nasm -o bootloader.o -g -f elf32 bootloader.asm
```

```
ld -o bootloader.symbol -melf_i386 -N bootloader.o -Ttext 0x7c00
```

5. 同实验任务一步骤，生成.bin 文件并写入磁盘

```
ld -o mbr.bin -melf_i386 -N mbr.o -Ttext 0x7c00 --oformat binary
```

```
ld -o bootloader.bin -melf_i386 -N bootloader.o -Ttext 0x7c00 --oformat binary
```

```
dd if=bootloader.bin of=hd.img bs=512 count=5 seek=1 conv=notrunc
```

```
dd if=mbr.bin of=hd.img bs=512 count=1 seek=0 conv=notrunc
```

6. 启动 qemu 调试

```
qemu-system-i386 -hda hd.img -s -S -parallel stdio -serial null
```

7. 启动 gdb 并远程连接 QEMU

```
gdb
```

```
target remote:1234
```

8. 加载符号表，加载 MBR 和 Bootloader 的符号表，以便在 GDB 中能够

正确显示源代码

```
add-symbol-file mbr.symbol 0x7c00
add-symbol-file bootloader.symbol 0x7e00

linhongyu@linhongyu-VirtualBox: ~/桌面/lab3/lab3.3

文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote:1234
Remote debugging using :1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x00000fff0 in ?? ()
(gdb) add-symbol-file mbr.symbol 0x7c00
add symbol table from file "mbr.symbol" at
.text_addr = 0x7c00
(y or n) y
Reading symbols from mbr.symbol...done.
(gdb) add-symbol-file bootloader.symbol 0x7e00
add symbol table from file "bootloader.symbol" at
.text_addr = 0x7e00
(y or n) y
Reading symbols from bootloader.symbol...done.
(gdb)
```

9. 打开调试窗口并设置断点

```
Layout src

break filename:linenum //在 filename 文件的 linenum 行设置断点

linhongyu@linhongyu-VirtualBox: ~/桌面/lab3/lab3.3

编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

告: 无法找到项目符号 _start; 缺省为 00000000
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
text 0x7c00 --oformat binary
告: 无法找到项目符号 _start; 缺省为 00000000
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
bootloader.o -Ttext 0x7c00 --oformat binary
告: 无法找到项目符号 _start; 缺省为 00000000
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
2 count=5 seek=1 conv=notrunc
+0 的读入
+0 的写出
tes (2.6 kB, 2.5 KiB) copied, 0.0001104 s
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
=1 seek=0 conv=notrunc
+0 的读入
+0 的写出
es copied, 0.00018867 s, 2.7 MB/s
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
parallel stdio -serial null
: Image format was not specified for 'h
Automatically detecting the format is
ns on block 0 will be restricted.
Specify the 'raw' format explicitly t
yu@linhongyu-VirtualBox:~/桌面/lab3/lab3.3
a2

linhongyu@linhongyu-VirtualBox: ~/桌面/lab3/lab3.3

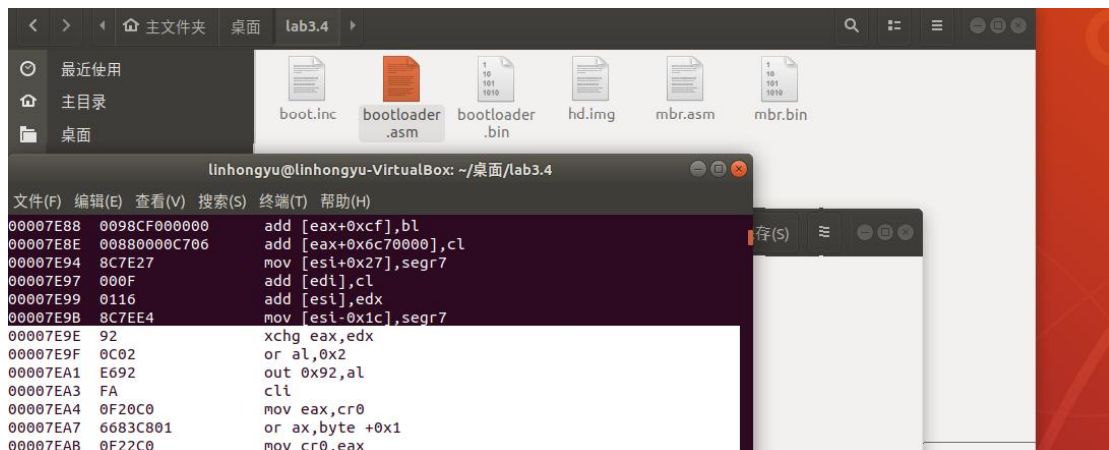
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)

bootloader.asm
1 %include "boot.inc"
2 [bits 16]
3
4 mov ax, 0xb800
5 mov gs, ax
6 mov ah, 0x03 ;青色
7 mov ecx, bootloader_tag_end - bootloader_tag
8 xor ebx, ebx
9 mov esi, bootloader_tag
10 output_bootloader_tag:
11 mov al, [esi]
12 mov word[gs:bx], ax
13 inc esi

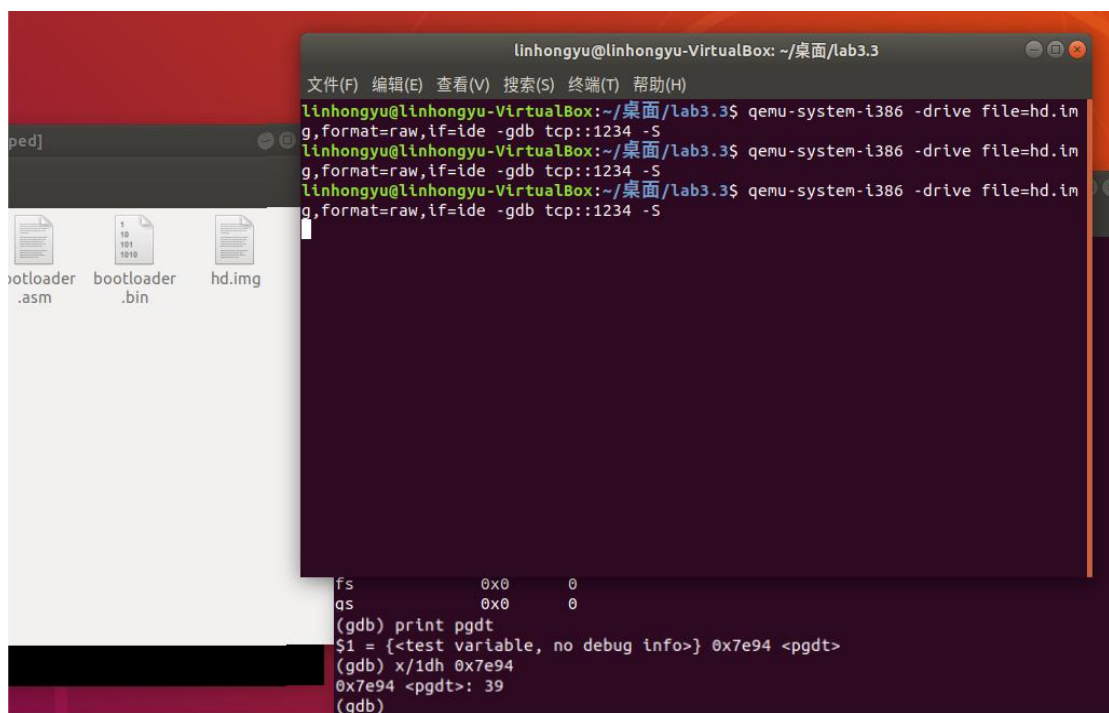
Remote Thread 1 In:
L4 PC: 0x7e00
(gdb) break *0x7e00
Breakpoint 1 at 0x7e00: file bootloader.asm, line 4.
(gdb) c
Continuing.
Breakpoint 1, ?? () at bootloader.asm:4
(gdb)
```

10. 除了使用调试串口，也可以进行反汇编，查看代码地址并设置相应断点

```
ndisasm -b 32 -o 0x7e00 bootloader.bin
```

11. 查看加载 GDTR，查看 pgdt 的值是否为 39



12. 查看第二十一根地址线


```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab3.4
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
For help, type "help".
Type "apropos word" to search for commands related to "word".
(gdb) target remote:1234
Remote debugging using :1234
warning: No executable has been specified and target does not support
determining executable automatically. Try using the "file" command.
0x0000fff0 in ?? ()
(gdb) break *0x7e9e
Breakpoint 1 at 0x7e9e
(gdb) break *0x7e9f
Breakpoint 2 at 0x7e9f
(gdb) break *0x7ea1
Breakpoint 3 at 0x7ea1
(gdb) x/1xh 0x92
0x92: 0x0000
(gdb) c
Continuing.

Breakpoint 1, 0x00007e9e in ?? ()
(gdb) x/1xh 0x92
0x92: 0xf000
(gdb) info registers al
al          0x2      2
```

13. 查看 CR0 寄存器的修改, 保护模式标志位
修改前 cr0 是 16, 修改后是 17

```
linhongyu@linhongyu-VirtualBox: ~/桌面/lab3.4
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
Continuing.

Breakpoint 5, 0x00007ea4 in ?? ()
(gdb) info register eax
eax          0xe88e   59534
(gdb) break* 0x7ea8
Breakpoint 7 at 0x7ea8
(gdb) info register eax
eax          0xe88e   59534
(gdb) c
Continuing.

Breakpoint 6, 0x00007ea7 in ?? ()
(gdb) info register eax
eax          0x10     16
(gdb) break* 0x7eae
Breakpoint 8 at 0x7eae
(gdb) c
Continuing.

Breakpoint 8, 0x00007eae in ?? ()
(gdb) info register eax
eax          0x11     17
```

14. 远跳转, 进入保护模式, 注意段寄存器

```
info registers
```

```
打开(O) ▾ g(gdb) info registers
out 0x92,al
cli
mov eax,cr0
or eax,1
mov cr0,eax
jmp dword COD
;16位的描述符
;流水线并串
[bits 32]
protect_mode_
(gdb) c
Continuing.
Breakpoint 2, 0x00007ee7 in ?? ()
(gdb) info registers
eax 0x365 869
ecx 0x12 18
edx 0xe88e 59534
ebx 0xa0 160
esp 0x7c10 0x7c10
ebp 0xd08e 0xd08e
esi 0x7eec 32492
edi 0xc031 49201
eip 0x7ee4 0x7ee4
eflags 0x6 [ PF ]
cs 0x20 32
ss 0x10 16
ds 0x8 8
es 0x8 8
fs 0x0 0
gs 0x18 24
(gdb) c
Continuing.
mov eax, DATA
mov ds, eax
mov es, eax
mov eax, STACK
mov ss, eax
mov eax, VIDEO_SELECT
mov gs, eax
mov ecx, protect_mode
mov ebx, 80 * 2
mov esi, protect_mode
mov ah, 0x3
mov al, 0x3
mov al,[esi]
mov [gs:ebx],ax
add ebx,byte +0x2
inc esi
loop 0x7ede
jmp short 0x7eea
```

- 实验结果展示：通过执行前述代码，可得下图结果。

实验结果在上面已经逐步呈现。

----- 实验任务 4 -----

- 任务要求：

在进入保护模式后，按照如下要求，编写并执行一个自己定义的 32 位汇编程序，要求简单说一说你的实现思路，并提供结果截图。使用两种不同的自定义颜色和一个自定义的起始位置(x,y)，使得 bootloader 加载后，在显示屏坐标(x,y)处开始输出自己的学号+姓名拼音首字母缩写，要求相邻字符前景色和背景色必须是相互对调的。公告图片中提供了学号为 21307233，姓名为宋小宝，自定义位置(12,12)的输出样式，仅供参考。代码实现框架可参考实验 2 和实验 3 的 MBR 程序。

- 思路分析：

为了实现这一功能，我们首先要确保已经成功进入保护模式并配置了视频段。接下来，我们编写汇编程序，利用 VGA 显存来直接操作屏幕显示内容。程序的主要思路：

初始化段寄存器：我们首先设置数据段、栈段和视频段寄存器。

设定显示起始位置：计算并设置字符的显示起始位置 (12, 12)。

设置前景色和背景色：为每个字符交替设置前景色和背景色。

输出学生信息：从字符串 student_id_name 中读取字符并显示在指定位置。

循环输出：直到所有字符输出完成。

- 实验步骤：给出关键部分的代码实现/命令行的执行过程，辅以必要的文字描述（可从指导书上摘抄）。

1. 主要部分代码编写

汇编代码，完成了进入保护模式后输出学号和姓名拼音首字母的功能

```
protect_mode_begin:
    ;设置数据段、栈段和视频段寄存器
    mov eax, DATA_SELECTOR      ;加载数据段选择子 (0..4GB)
    mov ds, eax                  ;将选择子加载到 ds
    mov es, eax                  ;将选择子加载到 es
    mov eax, STACK_SELECTOR      ;加载栈段选择子
    mov ss, eax                  ;将选择子加载到 ss
    mov eax, VIDEO_SELECTOR      ;加载视频段选择子
    mov gs, eax                  ;将选择子加载到 gs

    ;设置输出字符的起始位置
    mov esi, student_id_name     ;esi 指向学生 ID 的字符串
    mov ebx, (12 * 80 + 12) * 2   ;计算显示位置, 12 行 12 列, 80 列屏幕, 每个字符 2 字节 (16 位)
    ;计算学生 ID 字符串的长度
    mov ecx, student_id_name_end - student_id_name ;计算字符串长度
    mov edx, 0                   ;用于颜色切换的计数器 (切换前景和背景颜色)

output_student_info:
    mov al, [esi]                ;取字符
    test edx, 1                  ;检查颜色切换
    jz even_color                ;如果 edx 为偶数, 使用一个颜色
    mov ah, 0x1F                 ;奇数位置, 白色前景, 蓝色背景
    jmp set_char                 ;跳转到设置字符
even_color:
    mov ah, 0xF1                 ;偶数位置, 蓝色前景, 白色背景
set_char:
    mov word [gs:ebx], ax        ;将字符和颜色写入显存
    add ebx, 2                   ;移动到下一个字符位置 (每个字符占 2 字节)
    inc esi                      ;指向下一个字符
    inc edx                      ;增加计数器, 用于控制颜色切换
    loop output_student_info     ;循环直到字符串输出完
jmp $ ;进入死循环, 防止程序退出
student_id_name db '23320093LHY' ;学生 ID 字符串
student_id_name_end:            ;字符串结束标记
```

2. 思路分析

字符输出：通过指针 esi 指向学生学号和姓名拼音首字母的字符串，使用 gs 寄存器访问视频内存。每次读取一个字符后，我们交替设置前景色和背景色，

并将其写入视频内存中。

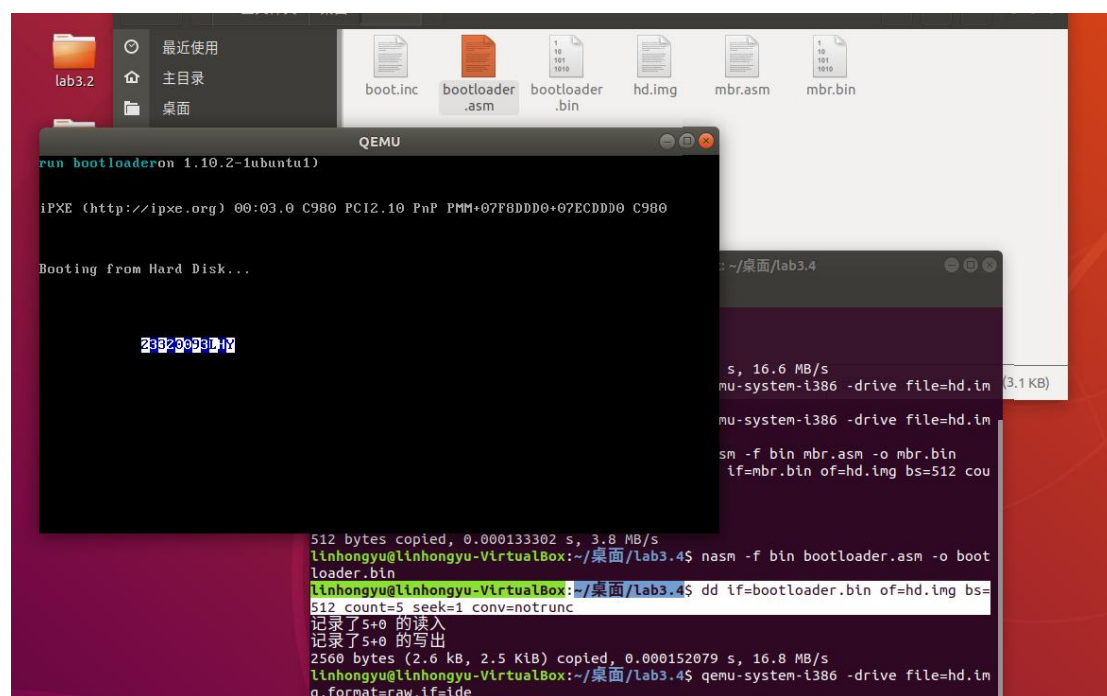
颜色切换：通过计数器 `edx` 来判断字符的奇偶位置，奇数位置使用白色前景蓝色背景，偶数位置使用蓝色前景白色背景。

位置计算：字符的起始位置由 $(12 * 80 + 12)$ 计算得出，表示在 80 列的屏幕中，从第 12 行第 12 列开始显示。

3. 编译、链接、写入磁盘、启动 QEMU、gdb 调试不再说明，同之前的实验任务

● 实验结果展示：通过执行前述代码，可得下图结果。

正确地输出了“23320093LHY”



Section 5 实验总结与心得体会

在本次实验中，我深入理解了保护模式的概念，并成功实现了在进入保护模式后在屏幕指定位置使用两种颜色显示学号和姓名拼音首字母的任务。在实验过程中，我遇到了一些关键问题，并通过查阅资料和调试逐步解决。首先，在 MBR 和 Bootloader 的加载过程中，我发现 Bootloader 并没有正确加载，导致进入保护模式的代码未被执行。经过调试，我确认是 `dd` 命令写入磁盘时 `seek` 参数不正确或 Bootloader 的起始地址计算出错。因此在进入保护模式后，用 GDT 正确设置了正确的选择字，并通过 GDB 以 `info registers` 命令检查正确加载。此外，在实现颜色交替显示时，开始导致奇数位置字符显示错误，调试发现是 `test edx, 1` 逻辑运算出错，或者 `edx` 没有正确更新。最后正确的微分项规划输出颜色，并通过转交选择字成功提供。通过此次实验，我更深刻地理解了操作系统的保护模式。

Section 6 对实验的改进建议和意见

本节为可选章节，针对指导书中一些表达不明确或者有错误的地方，可以在这里给出一些个人的建议和意见。

Section 7 附录：参考资料清单

1. <https://blog.csdn.net/xiaozhiwise/article/details/123247668>
2. https://blog.csdn.net/weixin_47195496/article/details/138069122
3. <https://www.cnblogs.com/mlzrq/p/10223060.html>
4. https://blog.csdn.net/qq_41922569/article/details/139206174

Section 8 附录：代码清单

压缩包中提供代码。